SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Concept Mapping

Course Code:	CSA0305	**Course Title:**	Data Structures
CO Assessed:	CO1 – Imitation (BL3, BL4)	**Assessment:**	Assessment Tool 2 – Concept Mapping
Weightage:	35% of CIA	**Total Marks:**	100
CO1 Statement:	*Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity.*		
Student Name:	K Suthesika	**Reg. No.:**	192511070
Section / Batch:	BE CSE	**Date:**	10/07/2026

Code of Conduct

I, K Suthesika (192511070) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K Suthesika

Instructions

- This test contains 5 Concept mapping questions. Total: 100 Marks.
- When a student answer using a concept map, in evaluation the answer should be with following four requirements: Understanding of concepts, Connections between ideas, Logical structure, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Linked data structure	Basic data structure	1	20
Primitive and Non-Primitive Data Structure - Linear and Non-Linear Data Structure	Classifications of Linear data structure	2	20
Search Data Structure	Linear Search and Binary Search	3	20
Recursion, Performance analysis	Recursion	4	20
Tree Data Structures	Classifications of Tree data structure	5	20
	GRAND TOTAL:	**100 Marks**	

Annexure A – Concept Mapping

1. A student attendance system records daily presence data. Construct a concept map linking Array → Fixed Size → Sequential Access → Linked List → Dynamic Allocation → Flexibility. Explain how each structure works and justify which is more suitable for dynamic data updates. (20 Marks)
2. A weather monitoring system collects temperature, humidity, and wind data continuously. Construct a concept map linking Primitive Data Types → Non-Primitive Structures → Linear Structures → Non-Linear Structures → Applications. Include examples and explain how each structure supports continuous data processing. (20 Marks)
3. A digital archive stores millions of documents. Construct a concept map linking Searching → Linear Search → Binary Search → Sorted Data → Time Complexity → Performance. Analyze the impact of data organization on search efficiency and justify the best technique. (20 Marks)
4. A program processes nested expressions using recursion and may encounter deep nesting. Construct a concept map linking Recursion → Call Stack → Stack Overflow → Error Handling → Efficiency. Analyze potential issues and justify strategies to handle them. (20 Marks)
5. A company maintains hierarchical employee records with multiple levels of management. Construct a concept map connecting Tree → Nodes → Levels → Traversal (DFS/BFS) → Searching → Efficiency. Analyze how traversal techniques affect performance and justify the most suitable approach for retrieving employee data. (20 Marks)

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

STUDENT ATTENDANCE SYSTEM

ARRAY

- Collection of elements stored in continuous memory locations.
- Suitable for storing attendance status of students.

FIXED SIZE

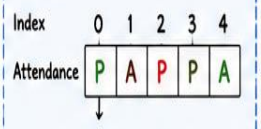
- Size is decided at the time of creation.
- Cannot grow or shrink during execution.

SEQUENTIAL ACCESS

- Elements are stored one after another.
- Accessed using index.
- Time Complexity: O(1)

HOW IT WORKS

- Data is stored in a linear manner.
- Direct access using index.



ADVANTAGES

- ✓ Fast access (O(1))
- ✓ Simple to implement
- ✓ Memory efficient for fixed data

DISADVANTAGES

- X Fixed size
- X Insertion/Deletion requires shifting elements
- X Resizing is expensive

LINKED LIST

- Collection of nodes.
- Each node contains data and address of next node.

DYNAMIC ALLOCATION

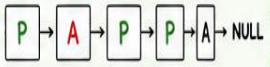
- Memory is allocated during program execution.
- Can grow or shrink as needed.

FLEXIBILITY

- Easy insertion and deletion of records.
- No need to shift elements.

HOW IT WORKS

- Data is stored in nodes.
- Each node points to the next node.



ADVANTAGES

- ✓ Dynamic size
- ✓ Easy insertion/deletion
- ✓ Memory used efficiently

DISADVANTAGES

- X Slower access (O(n))
- X Extra memory for pointers
- X More complex to implement

WHICH IS MORE SUITABLE FOR DYNAMIC DATA UPDATES?

If number of students is fixed and fast access by roll number is required → ARRAY is better.

If students may join, leave or records change frequently → LINKED LIST is better.

CONCLUSION

- ARRAY is best for fixed-size attendance records with frequent lookups.
- LINKED LIST is best for dynamic attendance systems where student records change often.

WEATHER MONITORING SYSTEM

Collects Temperature, Humidity, and Wind Data Continuously

1. PRIMITIVE DATA TYPES

Basic data types that store single values collected from sensors.

Examples

- int → Temperature (°C)
- float → Humidity (%)
- float → Wind Speed (m/s)
- char → Wind Direction (N,E,S,W)
- double → Pressure (hPa)
- bool → Rain Detected (Yes/No)

How it supports continuous data processing

- Primitive types store raw sensor readings efficiently in memory.
- Fast to read, write and process in real-time.

2. NON-PRIMITIVE STRUCTURES

Composite structures built using primitive types to store related data.

Examples

- struct WeatherRecord {int time; float temp; float humidity; float wind; char direction; float pressure; bool rain;}
- class SensorData { ... }

How it supports continuous data processing

- Groups multiple related readings into one record.
- Improves organization and manageability of streaming data.

3. LINEAR STRUCTURES

Data elements arranged in a sequential manner.

Examples

- **Array**
  Stores weather records at fixed time intervals.
- **Queue**
  Stores incoming real-time sensor data (FIFO).
- **Linked List**
  Stores data when size is unknown or dynamic.
- **Stack**
  Used for undo/redo operations in data processing.

How it supports continuous data processing

- Efficient for storing and accessing sequential sensor readings.
- Queue handles continuous incoming data without loss (FIFO).
- Linked List allows dynamic growth as data increases.

4. NON-LINEAR STRUCTURES

Data elements are not stored sequentially; have hierarchical or network relationships.

Examples

- **Tree** (Hierarchical)
  Organize data by location → Country → State → City → Weather Station.
- **Graph**
  Represent connections between multiple weather stations.
- **Hash Table**
  Store station ID and its latest weather data for fast lookup.

How it supports continuous data processing

- Trees help in hierarchical organization of large-scale weather data.
- Graphs analyze relationships and data flow between stations.
- Hash tables provide fast access to latest data for any station.

5. APPLICATIONS

Real-world use of data structures in weather monitoring system.

Examples

- Real-time dashboard for live weather display.
- Alert system for extreme conditions.
- Historical data analysis and forecasting.
- Data storage and retrieval for reports.

How it supports continuous data processing

- Enables efficient storage, retrieval and analysis of large volumes of streaming data.
- Supports quick decision-making and accurate forecasting.

CONCLUSION

From Primitive Data Types to Complex Structures, each step builds the foundation for efficient storage, organization, and real-time processing of continuous weather data for better monitoring and prediction.

DIGITAL ARCHIVE STORES MILLIONS OF DOCUMENTS

Efficient search is essential for quick retrieval of required documents.

1 SEARCHING

Process of finding a required document from a large collection.

Examples

- Search by Document ID
- Search by File Name
- Search by Keywords

Supports quick retrieval of records.

2 LINEAR SEARCH

Checks each document one by one until the target is found.

Example

Search through an unsorted archive.

Time Complexity

O(n)

Suitable for small or unsorted datasets.

3 BINARY SEARCH

Repeatedly divides the search space into two halves.

Example (Sorted Data)

10 20 30 40 50 60 70

Lower Half | Middle | Upper Half

10 20 30 40 50 60 70

10 20 30 | 50 60 70

10 20 30

Time Complexity

O(log n)

Very fast for large datasets.

4 SORTED DATA

Documents must be arranged in ascending or descending order before Binary Search can be applied.

Examples

- Sorted by Document ID
- Sorted by Date
- Sorted by File Name

Improves search efficiency.

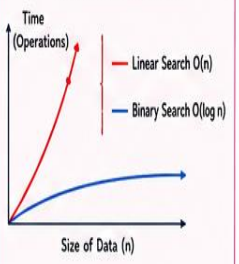
5 TIME COMPLEXITY

Measures how search time increases as data size (n) grows.

Comparison

Technique	Time Complexity
Linear Search	O(n)
Binary Search	O(log n)

Growth of Operations



6 PERFORMANCE

Efficient searching reduces response time and improves user experience.

Applications

- Digital Libraries
- Cloud Storage
- Government Archives
- Hospital Records

Faster access to information leads to better productivity.

IMPACT OF DATA ORGANIZATION ON SEARCH EFFICIENCY

✗ UNSORTED DATA

- No specific order of documents.
- Linear Search must be used.
- Time increases linearly with data size.
- Inefficient for large archives.

✓ SORTED DATA

- Documents arranged in order.
- Binary Search can be applied.
- Time increases logarithmically.
- Highly efficient for large archives.

KEY ANALYSIS

The better the data organization, the higher the search efficiency.

- Sorting has an initial cost O(n log n) but benefits millions of future searches.
- Essential for digital archives storing huge data.

BEST TECHNIQUE – JUSTIFICATION

Binary Search provides the highest performance for large digital archives when the data is sorted.

If the data is unsorted or changes frequently, Linear Search may be used until sorting is performed.

PROCESSING NESTED EXPRESSIONS USING RECURSION

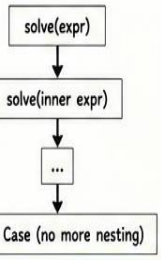
Concept Map: From Recursion to Efficiency

1. RECURSION

A function calls itself to solve smaller instances of a problem until a base case is reached.

Example – Nested Expression

Expression: ((a + b) * (c - d))



Why Useful?

- Naturally handles nested structures.
- Breaks complex problems into smaller similar subproblems.
- Easy to implement and understand.

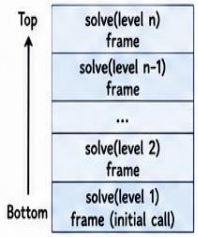
2. CALL STACK

Every recursive call is pushed onto the call stack with its own local variables and state.

How It Works

- Each call → a new stack frame.
- Stores: function parameters, local variables, return address.
- When base case returns, frames are popped in reverse order.

Visual (Call Stack)



Importance

Manages the control flow and keeps track of recursive calls.

3. STACK OVERFLOW

Occurs when the call stack exceeds its maximum size due to too many recursive calls (deep nesting).

Causes

- Very deep nesting in expressions (e.g., (((((a)))))).
- Missing or incorrect base case.
- Infinite recursion.
- Limited stack memory.

Example Scenario

For expression with 100,000+ nesting levels, each recursive call consumes stack space → stack limit exceeded → program crashes.

Result

Stack Overflow Error
(Abnormal Program Termination)

4. ERROR HANDLING

Detect, prevent, and handle stack overflow or excessive recursion to ensure program reliability.

Strategies

Set Maximum Depth Limit
Define a safe recursion depth limit. Stop recursion and report an error when limit is reached.

Validate Input
Check expression length and nesting level before processing.

Use Try-Catch / Exceptions
Catch stack overflow (when possible) and handle gracefully.

Convert to Iterative
Replace deep recursion with iterative approach (using an explicit stack).

Increase Stack Size
Adjust program/OS stack size if recursion depth is unavoidably large.

5. EFFICIENCY

Efficiency depends on managing recursion depth and memory usage effectively.

Positive Impact (When Handled Well)

- **Time Efficiency**
  Recursion simplifies code and often leads to better readability and maintainability.
- **Space Efficiency (Controlled)**
  With proper depth control, stack usage stays within safe limits.
- **Scalability**
  Optimized recursion or iterative solutions handle large inputs efficiently.

Negative Impact (If Not Handled)

- Excessive memory usage.
- Stack overflow → program crash.
- Poor performance due to repeated calls and large depth.

POTENTIAL ISSUES

- Deep nesting → many recursive calls.
- High stack memory consumption.
- Stack overflow → abrupt termination.
- Hard to debug when base case is incorrect.
- Performance degrades with very deep recursion.

JUSTIFICATION & BEST PRACTICES

- ✓ Always define and test base case correctly.
- ✓ Limit recursion depth and validate inputs.
- ✓ Prefer iterative approach for very deep nesting.
- ✓ Monitor stack usage during development.
- ✓ Use tail recursion optimization (if supported by language).

Example: Safer Approach

Instead of recursive parsing of nested expression, use a stack-based iterative parser to avoid stack overflow and improve reliability.

CONCLUSION

Recursion is powerful for processing nested expressions, but deep nesting can cause stack overflow. By understanding the flow from Recursion → Call Stack → Stack Overflow → Error Handling → Efficiency, we can build robust, efficient, and scalable programs.

KEY TAKEAWAY Use recursion wisely. Control the depth, handle errors gracefully, and optimize for efficiency to process even deeply nested expressions safely.

COMPANY EMPLOYEE RECORD SYSTEM

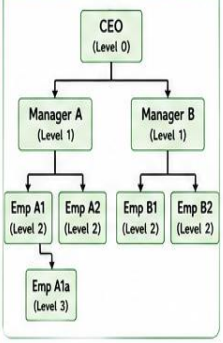
Concept Map: From Tree to Efficiency

Managing hierarchical employee data across multiple levels of management.

1. TREE

A tree is a non-linear data structure used to represent hierarchical relationships.

Example – Company Hierarchy



Key Points

- One root (topmost node – CEO)
- Each node can have zero or more children.
- Represents real-world hierarchies naturally.

2. NODES

Each element in a tree is called a node, storing employee information.

Node Structure

Employee ID
Name
Designation
Department
Pointer to Children

Types of Nodes

- **Root Node**: Topmost node (CEO)
- **Internal Node**: Has one or more children (Managers)
- **Leaf Node**: No children (Employees)
- **Sibling**: Nodes with the same parent

Nodes hold employee data and connection to subordinates.

3. LEVELS

Levels represent the depth of nodes in a tree, starting from the root.

Example (From Above Tree)

Level 0 : CEO
Level 1 : Manager A, Manager B
Level 2 : Emp A1, Emp A2, Emp B1, Emp B2
Level 3 : Emp A1a

Key Points

- Root is at Level 0.
- Children of a node are at next level.
- Helps in analyzing reporting depth and span of control.

More levels mean deeper hierarchy.

4. TRAVERSAL (DFS / BFS)

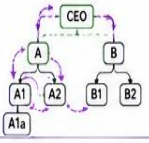
Traversal is the process of visiting all nodes in the tree in a specific order.

DFS – Depth First Search

Explores as deep as possible along each branch before backtracking.

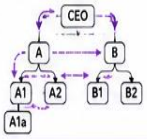
Types:

- Preorder (Root-Left-Right)
- Inorder (Left-Root-Right)
- Postorder (Left-Right-Root)



BFS – Breadth First Search

Visits all nodes level by level from left to right.



5. SEARCHING

Searching is used to find a specific employee (by ID, Name, Designation, etc.) in the tree.

Search using Traversal

- **DFS Search:** Follows one branch deeply, may take more time if target is far from root.
- **BFS Search:** Checks level by level, usually finds closer (higher level) employees faster.

Example Search: Find 'Emp B2'

- DFS (Preorder): CEO → A → A1 → A1a → A2 → B → B1 → B2 (Found)
- BFS (Level Order): CEO → A → B → A1 → A2 → B1 → B2 (Found earlier)

Choice of traversal impacts search time.

6. EFFICIENCY

Efficiency depends on tree structure, traversal method, and size of data.

Comparison

Aspect	DFS	BFS
Time Complexity	O(n)	O(n)
Space Complexity	O(h) (stack)	O(w) (queue)
Best For	Deep data exploration	Finding nearest/ level-wise data
Memory Usage	Low (uses stack)	High (uses queue)
Finds Closer Node Faster?	No	Yes

Key Note

n = number of nodes
h = height of tree
w = maximum width

IMPACT OF TRAVERSAL ON PERFORMANCE

DFS Impact

- ✓ Good for scenarios where data is spread deep in hierarchy.
- ✓ Uses less memory (stack based).
- ✗ May take longer to find a node at higher or other branches.

BFS Impact

- ✓ Good for finding employees at higher levels.
- ✓ Returns nearest match in fewer steps.
- ✗ Uses more memory on wide trees.

When to Use?

- Use **DFS** when you need to explore entire subtree, analyze hierarchy, or perform operations like delete, copy, or path finding.
- Use **BFS** when you need to find a specific employee quickly, especially if they are near the top levels.

JUSTIFICATION – MOST SUITABLE APPROACH

For retrieving employee data (searching by ID, Name, or Position), BFS (Level Order Traversal) is the most suitable approach because:

- It finds employees at higher levels faster.
- It guarantees the shortest path in an unweighted tree.
- It is ideal for organizational hierarchies where top management data is requested more frequently.

CONCLUSION

Trees efficiently represent hierarchical employee data. Traversal techniques (DFS/BFS) directly affect search performance. For quick retrieval of employee information, BFS is preferred, while DFS is better for deep analysis and processing tasks.